Design Airbnb

SystemsExpert design walkthrough

Clarifying questions

- Search + booking + host listing management in scope? Payments?
- Can two guests book overlapping dates? (No)
- Scale of listings?

Functional requirements

- Hosts create listings with availability
- Guests search by location/dates/price
- Guests book; double booking impossible

Non-functional requirements

- Search fast (< 300 ms)
- Booking strongly consistent

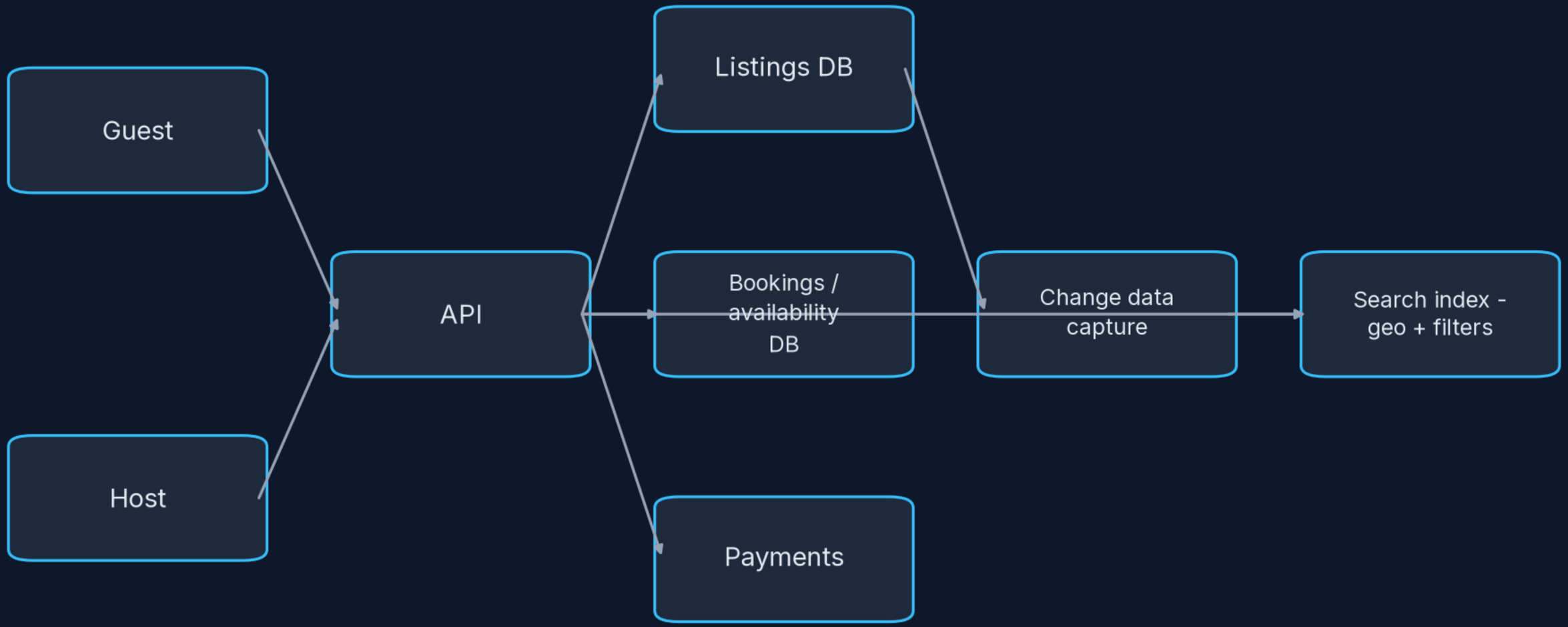
Capacity estimates

- ~10M listings, ~100M searches/day (~1.2k/s), ~1M bookings/day

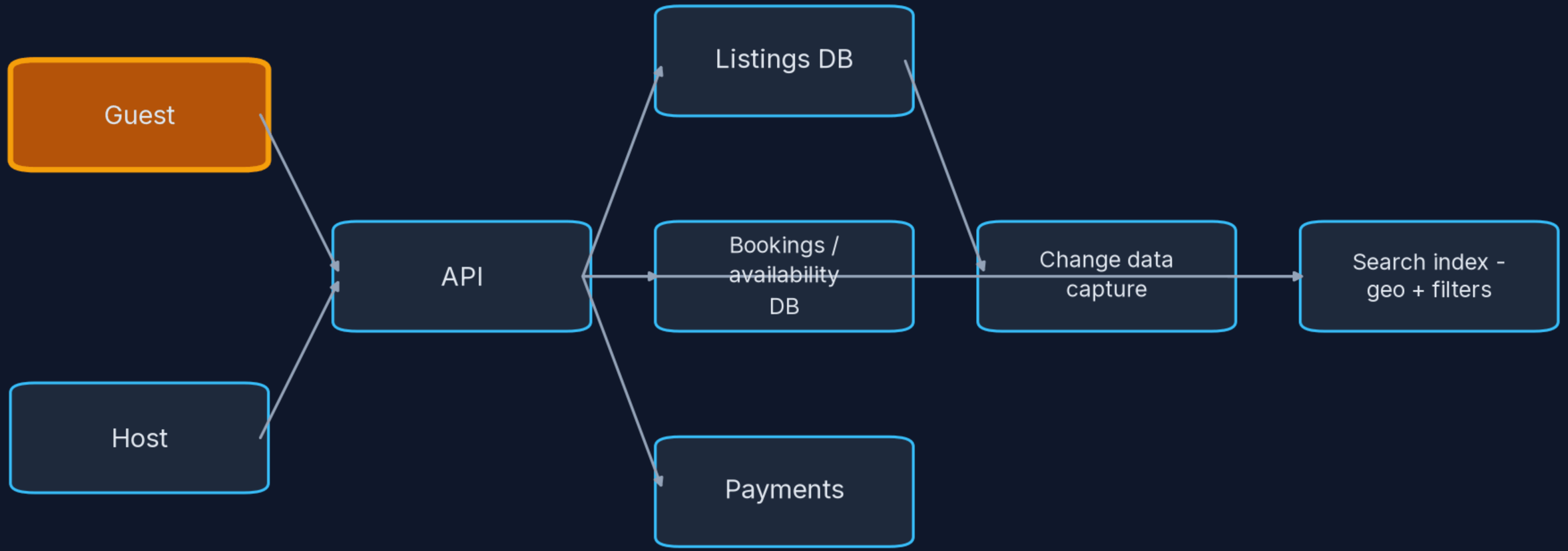
Capacity: worked out

- 100M searches/day ÷ 86,400 ≈ 1.2k/s
- Peak ~3× ≈ 3.5k searches/s
- 1M bookings/day ≈ 12/s
- Searches ≫ bookings → optimise reads, make writes correct

High-level architecture

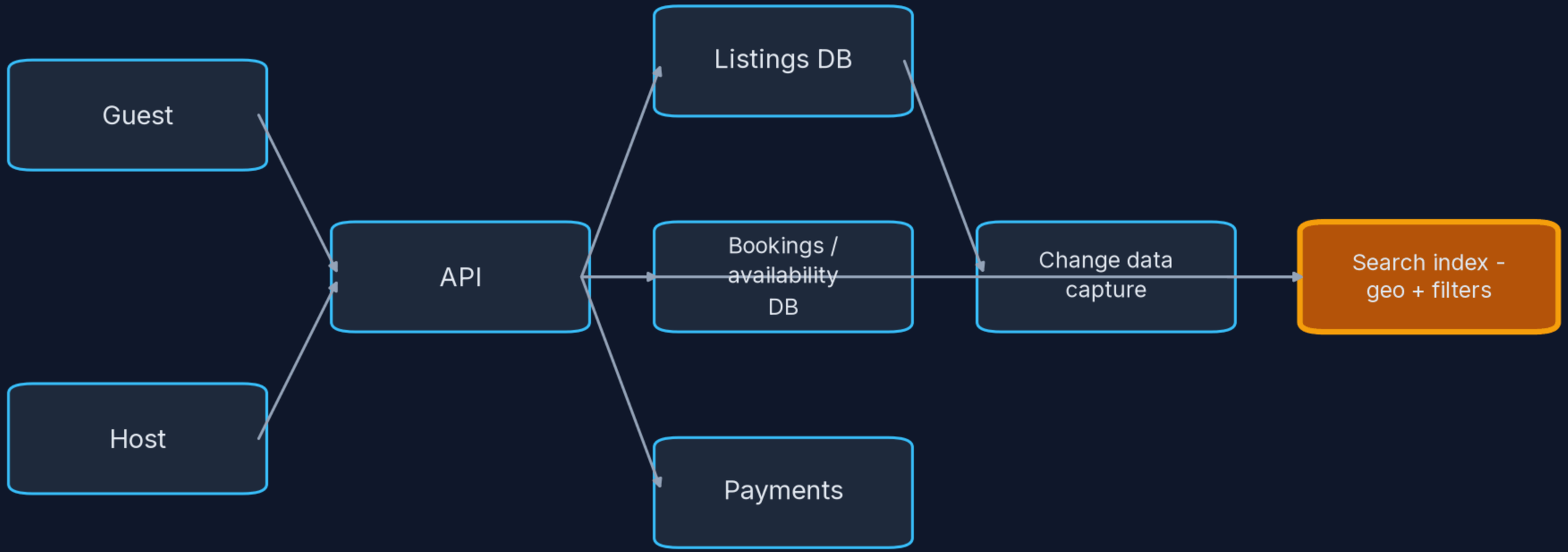


Request flow: Book a stay



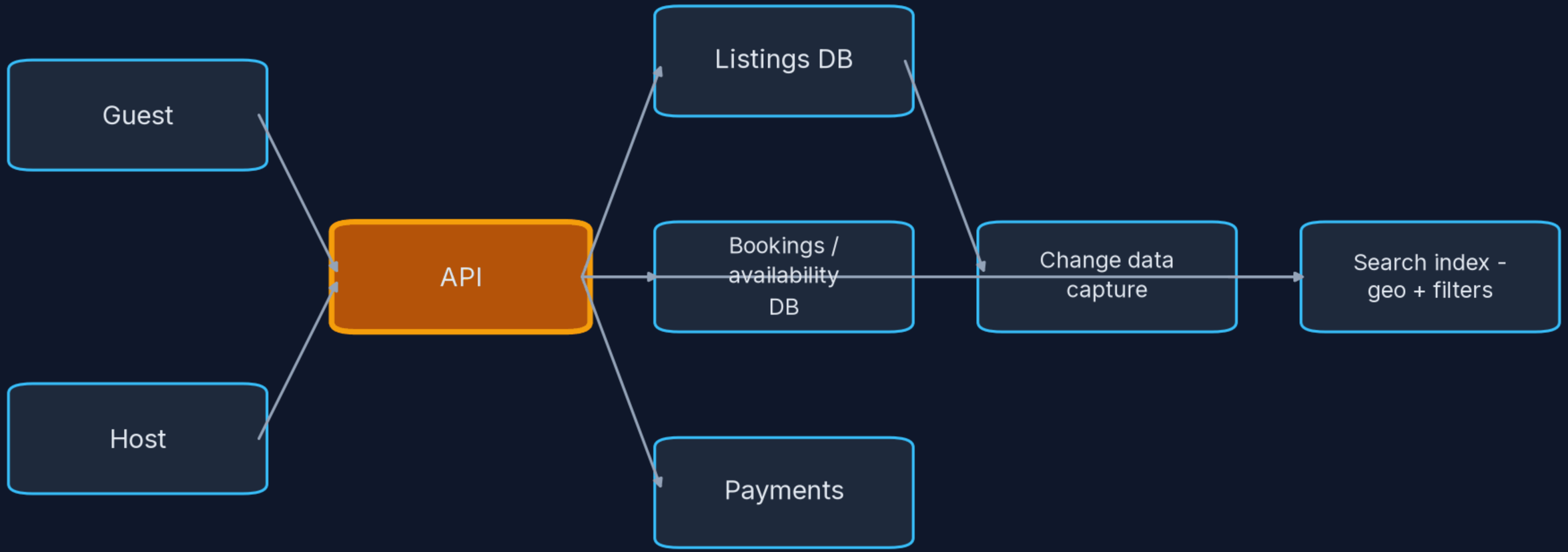
1. Guest searches

Request flow: Book a stay



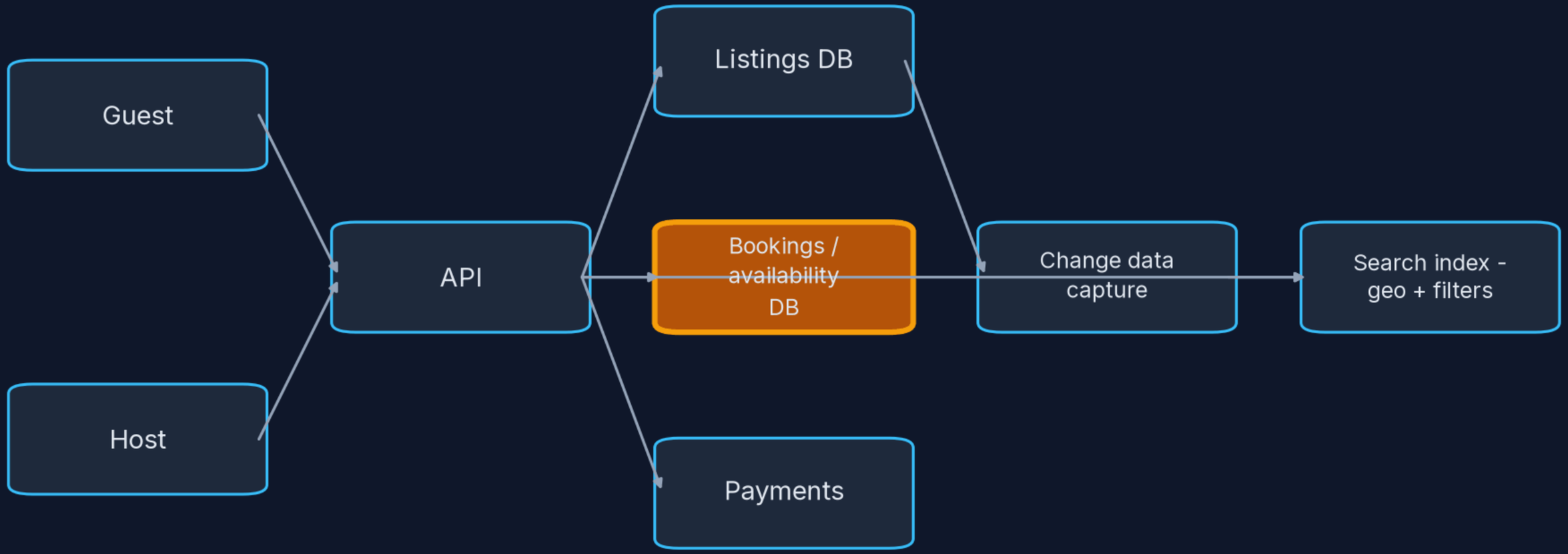
2. Geo search

Request flow: Book a stay



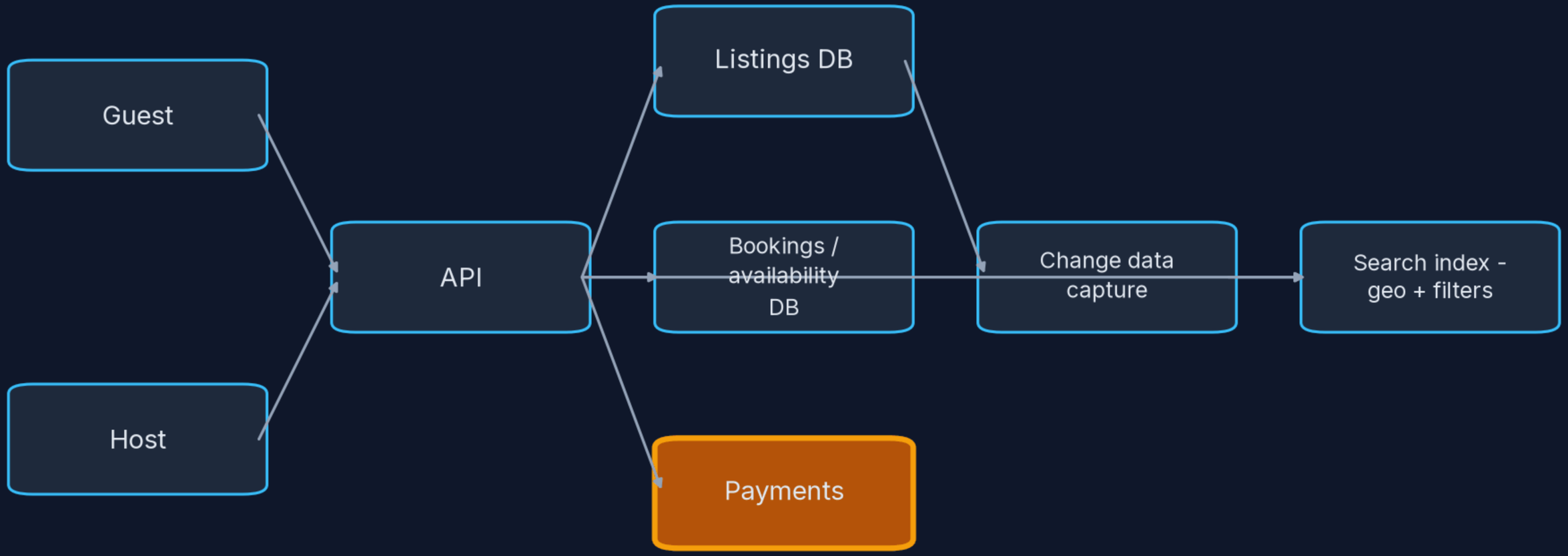
3. Guest books a listing

Request flow: Book a stay



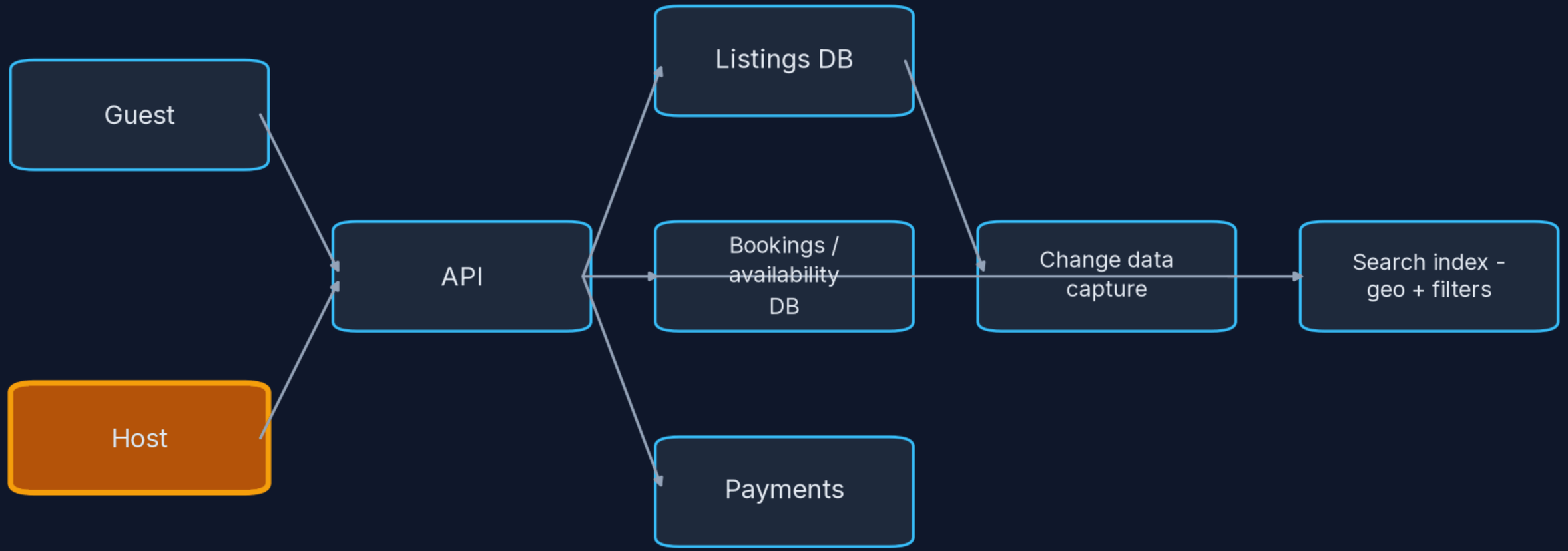
4. Reserve the dates atomically

Request flow: Book a stay



5. Charge the guest

Request flow: Book a stay



6. Host is notified

Data model

- Listings(id, host_id, geo, price, attributes)
- Availability(listing_id, date, status) PK(listing_id, date)
- Bookings(id, listing_id, guest_id, start, end, status)

API

```
GET  /search?lat&lng&radius&checkIn&checkOut&guests&priceMax
GET  /listings/{id}
POST /listings  (host)
POST /bookings {listingId, checkIn, checkOut, idempotencyKey}
```

No double booking

- In one transaction, lock the availability rows for the date range (SELECT … FOR UPDATE) and mark them reserved; or insert into a table with a unique (listing_id, date) constraint.
- Temporary holds expire if payment fails.

Search

- Geo index (geohash/quadtree) in Elasticsearch with date availability filters; results may be slightly stale, so booking re-validates.

Payments

- Authorise at booking, capture after check-in.

Key decisions

- Separate search index (CDC) → fast geo and filter queries
- DB constraint on date overlap → no double booking
- Search can be slightly stale → availability rechecked at booking
- Idempotent booking requests → safe retries

Trade-offs & alternatives

- Search eventual consistency vs booking strong consistency
- Per-date rows are simple; range types/interval trees are more compact

Recap

- Clarify scope, then functional + non-functional needs
- Estimate load to size storage and fan-out
- Core pieces: Guest, API, Host, Listings DB, Change data capture
- Know your trade-offs and failure modes